DELOITTE FULL STACK DEVELOPMENT TRAINING PROGRAM

Trainer: Vishwas K Singh, SME-FSD | CloudThat

EVENTZEN

Event Management System

Capstone Project – Full Stack Development

Submitted By	**Bhavya Mittal**
Program	Deloitte FSD Training Program
Trainer	**Vishwas K Singh**
Date	March 30, 2026
GitHub	**`github.com/Bhavya333-web/eventzen`**

TABLE OF CONTENTS

Section	Title	Page
1	Project Overview & Problem Statement	3
2	System Architecture	5
3	Technology Stack	6
4	Module 1 – User Authentication	8
5	Module 2 – Event Management	11
6	Module 3 – Attendee Management	14
7	Module 4 – Budget Tracking	17
8	Module 5 – Vendor Management	19
9	ER Diagram & Database Schema	21
10	Complete API Endpoints Reference	23
11	Frontend Pages & User Interface	26
12	User Flow Diagrams	30
13	Docker & Deployment Configuration	32
14	Security Implementation	35
15	Testing & Quality Assurance	37
16	Project Structure & Repository	45

1. PROJECT OVERVIEW & PROBLEM STATEMENT

1.1 Introduction

EventZen is a comprehensive, enterprise-grade Event Management Platform developed as the Capstone Project for the Deloitte Full Stack Development Training Program. The platform is architected on a modern Microservices Architecture and provides end-to-end event management capabilities for event planning companies. The system supports two distinct user roles – Administrators who manage the entire platform, and Regular Customers who discover and book events through a dedicated customer portal.

1.2 Problem Statement

An event planning company, EventZen, was struggling to manage its growing number of events efficiently. The following critical business challenges were identified and addressed by this system:

Challenge	Description	Impact
Manual Event Scheduling	Time-consuming, error-prone scheduling of events, venue bookings, and vendor coordination	High – delays and double bookings
Inefficient Attendee Management	No system for tracking registrations, managing guest lists, or preventing duplicate bookings	High – poor attendee experience
Complex Budget Tracking	No centralized way to manage event budgets, categorized expenses, or financial reports	Medium – overspending risk
Limited Customer Engagement	No user-facing platform for customers to view events, book tickets, or track their registrations	High – lost business

1.3 Solution Summary

EventZen addresses all four challenges through a fully integrated, microservices-based web application with five functional modules:

Module	Technology	Core Functionality
User Authentication	Node.js + MySQL + JWT	Registration, login, role-based access control
Event Management	Node.js + MongoDB	Full CRUD events with status tracking
Attendee Management	.NET 8 + MongoDB	Registration, listing, duplicate prevention

Module	Technology	Core Functionality
Budget Tracking	Spring Boot + MongoDB	Budget creation, expense categorization
Vendor Management	Node.js + MongoDB	Vendor directory, event assignment

1.4 Project Objectives

- Build a production-ready microservices application with 5 independent, containerized services
- Implement secure JWT-based authentication with Role-Based Access Control (Admin / User)
- Deliver at least 4 functional modules excluding authentication, as per the capstone requirement
- Achieve full Docker containerization of all services with Docker Compose orchestration
- Implement polyglot persistence using MySQL (relational) and MongoDB (NoSQL) strategically
- Create a responsive React.js SPA with separate Admin Portal and Customer Portal
- Follow RESTful API design standards with proper HTTP verbs and status codes
- Produce complete documentation including ER diagram, wireframes, user flow, and API reference

2. SYSTEM ARCHITECTURE

2.1 Architectural Pattern

EventZen adopts a Microservices Architectural Pattern where each major business domain is implemented as an independent service with its own database, programming language/framework, and deployment unit. This ensures loose coupling, high cohesion, independent scalability, and technology diversity. All services communicate through synchronous RESTful HTTP APIs and are orchestrated via Docker Compose for unified deployment within a shared bridge network.

2.2 Architecture Overview

```
                React Frontend  (Port 3000)  -  Admin Portal  +  Customer Portal
                                |  HTTP REST Calls  |
    +------------------+------------------+-----------------+
    |                  |                  |                 |
Auth Service (5001)  Event Service (5002)  Attendee Svc (5172)  Budget Svc (5003)
Node.js + MySQL      Node.js + MongoDB    .NET 8 + MongoDB     Spring Boot + MongoDB
```

2.3 Service Details

Service	Container Name	Port	Technology	Database	Responsibility
Auth Service	eventzen-auth	5001	Node.js + Express.js	MySQL 8.0	JWT Auth & RBAC
Event Service	eventzen-events	5002	Node.js + Express.js	MongoDB 6.0	Events & Vendors CRUD
Attendee Service	eventzen-attendees	5172	.NET 8 / ASP.NET Core	MongoDB 6.0	Attendee Registration
Budget Service	eventzen-budget	5003	Spring Boot / Java 21	MongoDB 6.0	Budget & Expenses
Frontend	eventzen-frontend	3000	React.js 18	N/A	Web SPA (2 portals)
MongoDB	eventzen-mongodb	27017	MongoDB 6.0	N/A	NoSQL database
MySQL	eventzen-mysql	3307 (host)	MySQL 8.0	N/A	Relational database

3. TECHNOLOGY STACK

3.1 Complete Technology Matrix

Layer	Technology	Version	Purpose
Frontend	React.js	18.x	Single Page Application – primary UI framework
Frontend	Framer Motion	Latest	Smooth animations and page transitions
Frontend	Recharts	Latest	Data charts on admin dashboard
Frontend	Axios	Latest	HTTP client for API communication
Frontend	React Context API	Built-in	Authentication state management
Auth Backend	Node.js	18+	JavaScript runtime environment
Auth Backend	Express.js	4.x	Web application framework
Auth Backend	jsonwebtoken	9.x	JWT creation and verification
Auth Backend	bcrypt	5.x	Secure password hashing
Auth Backend	mysql2	3.x	MySQL database driver
Event Backend	Node.js	18+	JavaScript runtime environment
Event Backend	Express.js	4.x	Web application framework
Event Backend	Mongoose	7.x	MongoDB Object Document Mapper
Attendee Backend	ASP.NET Core	.NET 8.0	Cross-platform Web API framework
Attendee Backend	MongoDB.Driver	2.x	Official MongoDB C# driver
Budget Backend	Spring Boot	3.x	Java application framework
Budget Backend	Java	21 LTS	Backend programming language

Layer	Technology	Version	Purpose
Budget Backend	Spring Data MongoDB	3.x	MongoDB integration for Spring
Database	MySQL	8.0	Relational DB – user authentication data
Database	MongoDB	6.0	NoSQL DB – events, attendees, budget, vendors
DevOps	Docker	Latest	Container platform
DevOps	Docker Compose	3.8	Multi-container orchestration
Dev Tools	Postman	Latest	API testing and documentation
Version Control	Git / GitHub	Latest	Source code management

3.2 Why Polyglot Persistence?

EventZen deliberately uses two different database systems, demonstrating real-world enterprise decision-making:

- MySQL for Authentication – User credentials require ACID compliance, strong consistency, and relational integrity constraints. MySQL enforces email uniqueness via UNIQUE constraints and supports transactional guarantees essential for secure user management.
- MongoDB for All Other Domains – Event, attendee, budget, and vendor data have flexible and evolving schemas. MongoDB's document model matches the JSON-native nature of this data, offers horizontal scalability, and avoids complex JOIN operations.

4. MODULE 1 – USER AUTHENTICATION

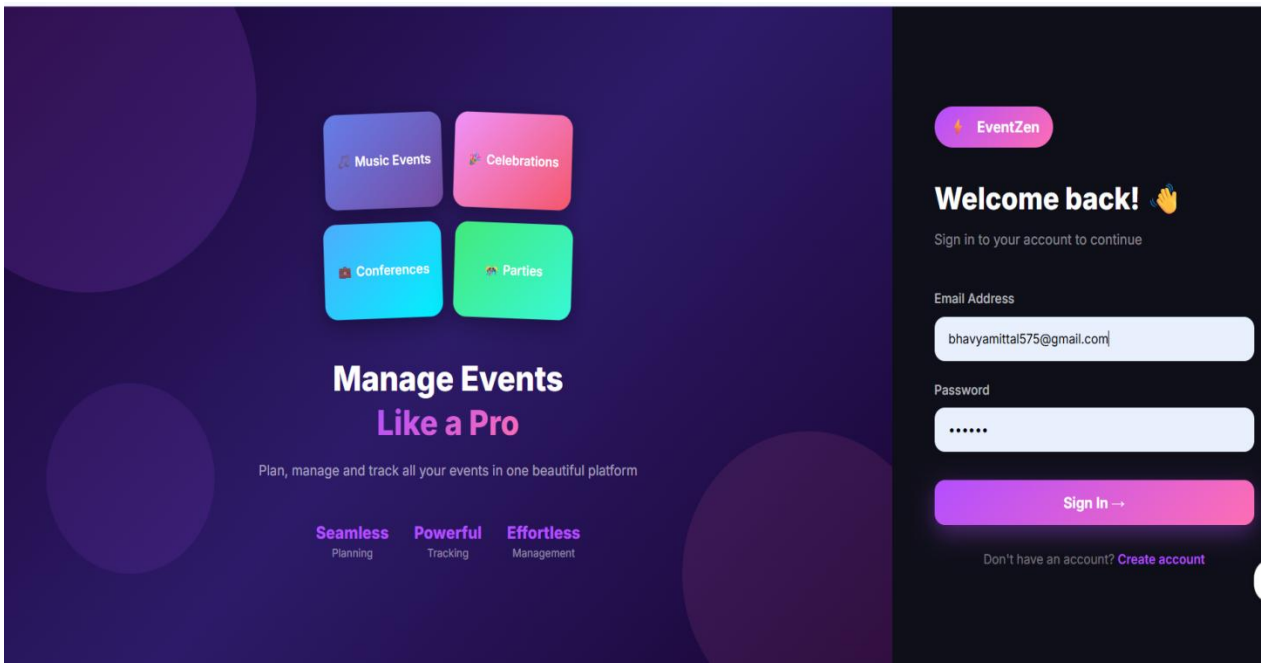
4.1 Module Overview

The User Authentication Module is the security foundation of EventZen. Built with Node.js and Express.js and backed by MySQL 8.0, it handles all aspects of user identity, access control, and session management. The module supports two distinct user roles – Admin and Regular User – each with a dedicated portal and granular access permissions. All passwords are hashed using bcrypt and sessions are secured via JSON Web Tokens (JWT).

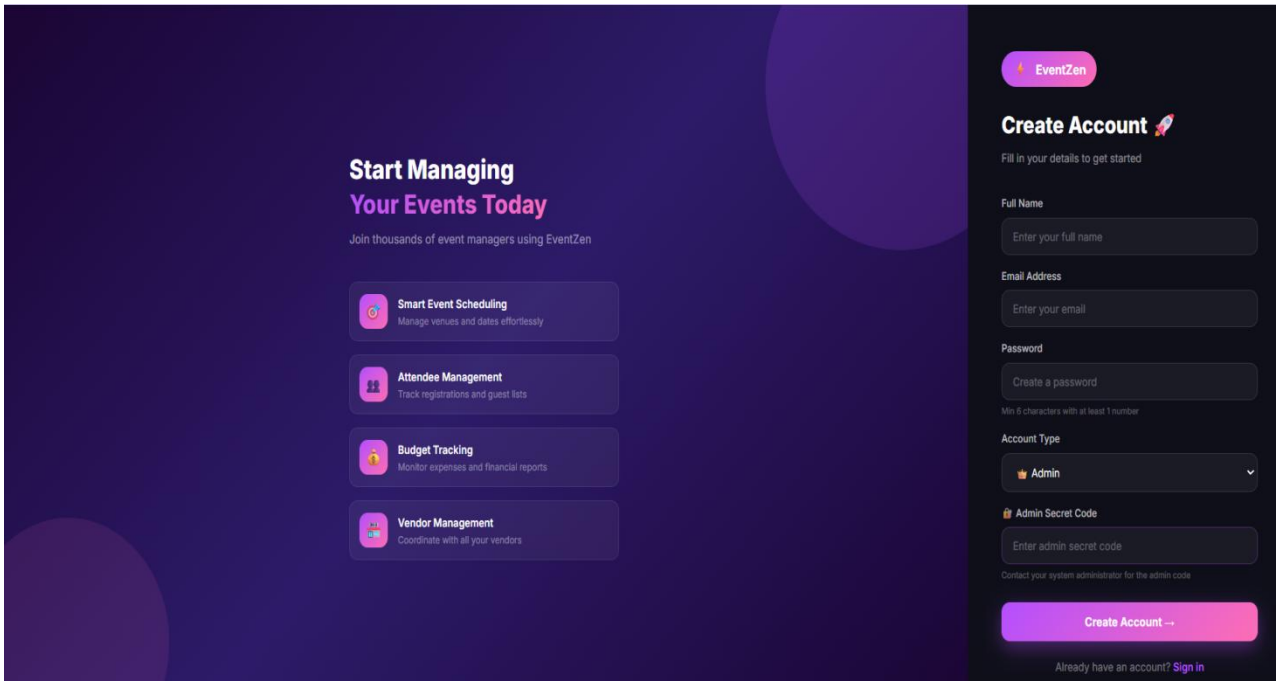
4.2 Key Features

- User registration with full name, email, password, and role selection (Admin or Regular User)
- Admin Secret Code protection – only users with code EVENTZEN@ADMIN2025 can create admin accounts
- JWT-based stateless authentication with token expiry and middleware validation on protected routes
- Password hashing using bcrypt with salt rounds – passwords are never stored in plaintext
- Role-Based Access Control separating Admin Portal (/dashboard) from Customer Portal (/user)
- Password validation – minimum 6 characters with at least one numeric digit
- Admin user management endpoint to list all registered platform users
- Client-side token storage with automatic redirect based on decoded user role

Login Page



Register Page



4.3 MySQL Database Schema – eventzen_auth

Table: users

Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)	NOT NULL	User full name
email	VARCHAR(100)	UNIQUE, NOT NULL	Login email address
password	VARCHAR(255)	NOT NULL	bcrypt-hashed password (never plaintext)
role	VARCHAR(20)	DEFAULT 'user'	Role: 'admin' or 'user'
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

4.4 API Endpoints – Auth Service (Base URL: http://localhost:5001)

#	Method	Endpoint	Description	Auth Required	Response Code
1	POST	/api/auth/register	Register new user (Admin or User)	No	201 Created

#	Method	Endpoint	Description	Auth Required	Response Code
2	POST	/api/auth/login	Login and receive JWT token	No	200 OK
3	GET	/api/auth/users	Retrieve all registered users	Yes (Admin JWT)	200 OK

4.5 Request / Response Examples

POST /api/auth/register – Admin Account Registration

Request Body:

```
{ "name": "Bhavya Mittal", "email": "admin@eventzen.com",
  "password": "Admin@123", "role": "admin", "adminCode": "EVENTZEN@ADMIN2025" }
```

Response (201 Created):

```
{ "message": "User registered successfully" }
```

POST /api/auth/login – Invalid Credentials

Request Body:

```
{ "email": "admin@eventzen.com", "password": "wrongpassword" }
```

Response (400 Bad Request):

```
{ "message": "Wrong password" }
```

4.6 JWT Middleware Flow

1. Client sends POST /api/auth/login with email and password
2. Auth Service queries MySQL to find user record by email
3. bcrypt.compare() verifies the submitted password against stored hash
4. On success, jwt.sign() generates a signed token containing userId and role
5. Token returned to client and stored in localStorage
6. Subsequent requests include token in Authorization: Bearer <token> header
7. JWT middleware verifies token signature and decodes payload on each protected route
8. Role check determines whether Admin or User portal endpoints are accessible

5. MODULE 2 – EVENT MANAGEMENT

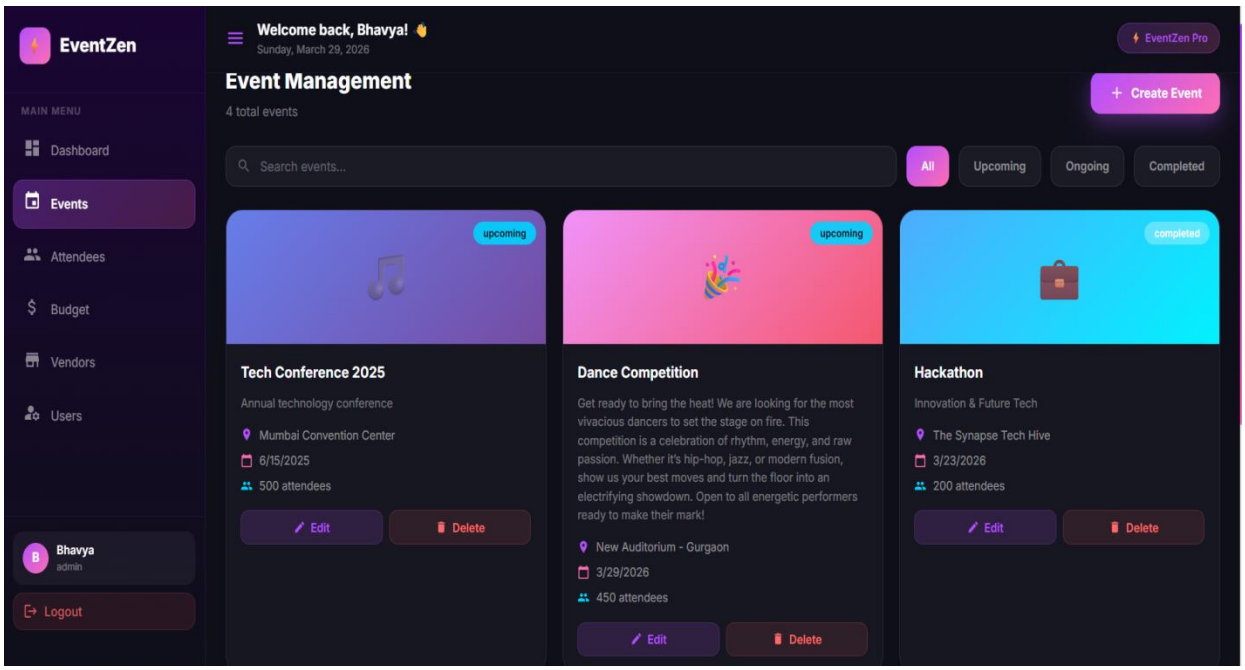
5.1 Module Overview

The Event Management Module is built on Node.js and Express.js with MongoDB as the persistence layer. This module forms the operational core of EventZen, enabling administrators to create, manage, and track events through their complete lifecycle. Events progress through three status states: Upcoming, Ongoing, and Completed. The module also encompasses Vendor Management as a closely related subdomain, allowing vendors to be created and assigned to specific events.

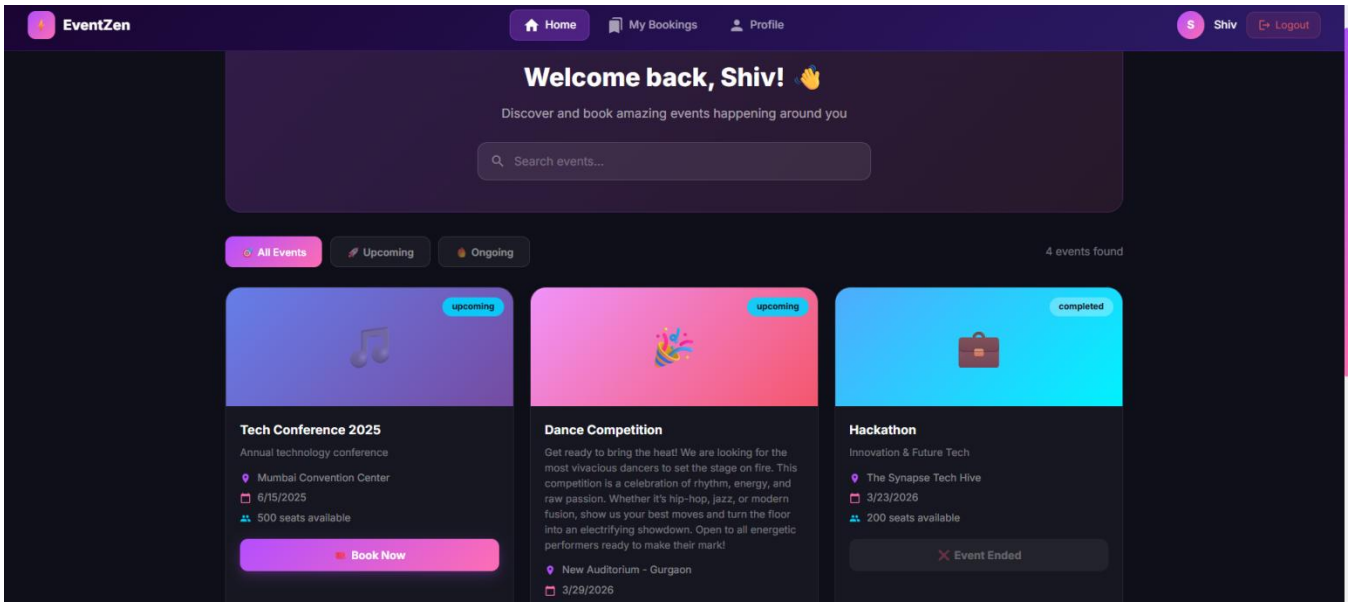
5.2 Key Features

- Create events with title, description, date, venue, capacity, and initial status
- Read and list all events with search by title, filter by status (upcoming/ongoing/completed)
- Update event details including title, venue, date, capacity, and status progression
- Delete events with full removal from MongoDB
- Event status lifecycle tracking: Upcoming → Ongoing → Completed
- Venue and capacity management with real-time seat availability display
- Customer portal discovery view showing available events with Book Now button
- Vendor CRUD operations – create, view, update, delete vendor records
- Vendor categorization: Catering, Decoration, Entertainment, Photography, Logistics, AV/Tech
- Assign vendors to specific events via eventId foreign reference

Event Page(Admin)



Event Page(Customer)



5.3 MongoDB Schema – Events Collection (eventzen_events database)

Field	Type	Required	Description
_id	ObjectId	Auto	MongoDB primary key (auto-generated)
title	String	Yes	Name of the event
description	String	No	Detailed event description
date	Date	Yes	Event date and time
venue	String	Yes	Event location or venue name
capacity	Number	No	Maximum number of attendees
status	String (Enum)	No	upcoming / ongoing / completed (default: upcoming)
createdAt	Date	Auto	Record creation timestamp (Mongoose timestamps)
updatedAt	Date	Auto	Last update timestamp (Mongoose timestamps)

5.4 MongoDB Schema – Vendors Collection

Field	Type	Required	Description
_id	ObjectId	Auto	MongoDB primary key

Field	Type	Required	Description
eventId	ObjectId	Yes	Foreign reference to Events._id
name	String	Yes	Vendor company or individual name
category	String	No	Catering / Decoration / Entertainment / Photography / Logistics / AV-Tech / Other
phone/email	String	No	Vendor contact information
price	Number	No	Vendor pricing for the event
notes	String	No	Additional vendor notes

5.5 API Endpoints – Event Service (Base URL: http://localhost:5002)

#	Method	Endpoint	Description	Auth	Code
1	GET	/api/events	Get all events	No	200
2	GET	/api/events/:id	Get single event by ID	No	200
3	POST	/api/events	Create a new event	JWT	201
4	PUT	/api/events/:id	Update event by ID	JWT	200
5	DELETE	/api/events/:id	Delete event by ID	JWT	200
6	GET	/api/vendors	Get all vendors	No	200
7	GET	/api/vendors/:id	Get single vendor	No	200
8	POST	/api/vendors	Create new vendor	JWT	201
9	PUT	/api/vendors/:id	Update vendor by ID	JWT	200
10	DELETE	/api/vendors/:id	Delete vendor by ID	JWT	200

6. MODULE 3 – ATTENDEE MANAGEMENT

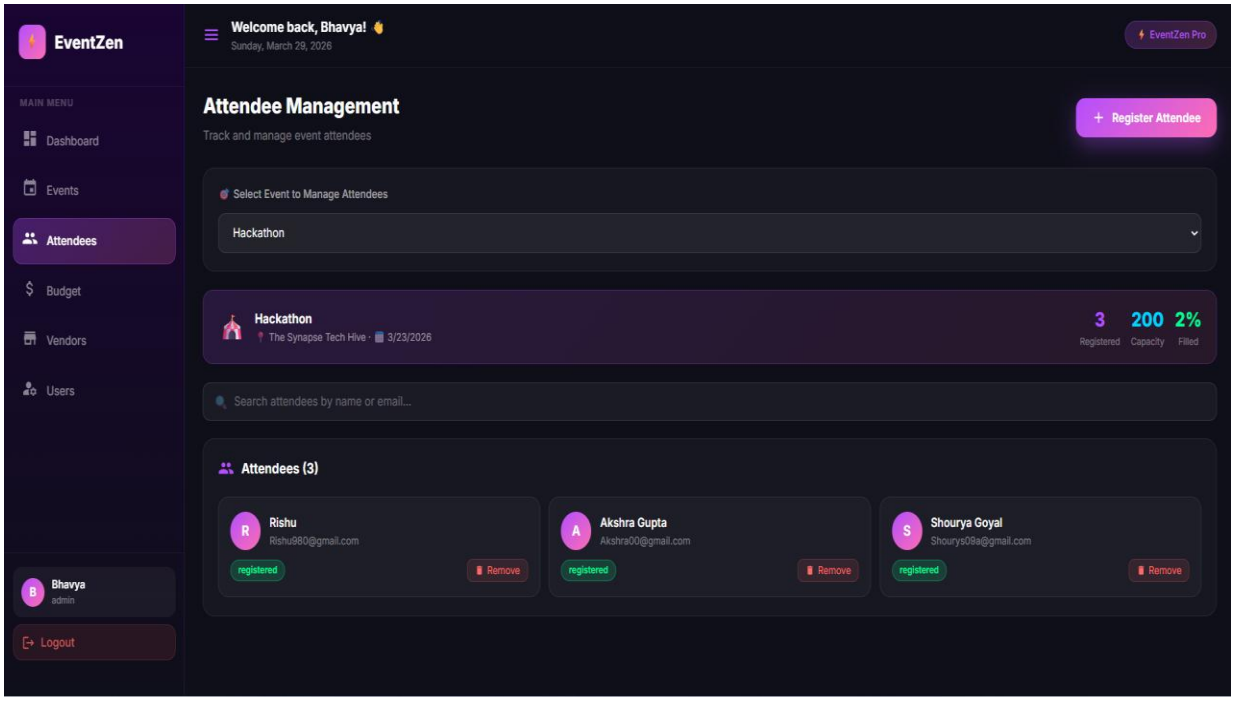
6.1 Module Overview

The Attendee Management Module is implemented as an independent microservice using ASP.NET Core Web API running on .NET 8. This is the only service in the stack built on the Microsoft .NET platform, demonstrating the polyglot nature of the microservices architecture. The service handles all attendee-related operations and includes built-in duplicate booking prevention to ensure data integrity.

6.2 Key Features

- Register an attendee by providing name, email, and target eventId
- Retrieve complete attendee roster for any given event using the eventId
- Prevent duplicate registrations – same email address cannot register twice for the same event
- Remove individual attendee records using unique attendee ID
- Automatic status assignment ('registered') on successful registration
- Automatic registeredAt timestamp recorded on every successful booking
- Admin attendee management view with per-event filtering

Attendee Management Page



6.3 MongoDB Schema – Attendees Collection (eventzen_attendees database)

Field	Type	Required	Description
id	ObjectId	Auto	MongoDB _id returned as 'id' in response
name	String	Yes	Attendee full name
email	String	Yes	Attendee email address
eventId	String	Yes	Reference to Events._id (stored as string)
status	String	Auto	Always 'registered' on creation
registeredAt	DateTime	Auto	UTC timestamp of registration (ISO 8601)

6.4 API Endpoints – Attendee Service (Base URL: http://localhost:5172)

#	Method	Endpoint	Description	Auth Required	Code
1	GET	/api/attendee/event/{eventId}	Get all attendees for an event	No	200
2	POST	/api/attendee	Register a new attendee	No	200
3	DELETE	/api/attendee/{id}	Remove an attendee by ID	JWT (Admin)	200

6.5 Sample API Request & Response

POST /api/attendee – Register Attendee

Request Body:

```
{ "name": "Bhavya Mittal", "email": "bhavya@gmail.com",
  "eventId": "69c915c9e2dfc767e86d5ed9" }
```

Response (200 OK):


```
{ "id": "69c9181d21031e5101d10571",
  "name": "Bhavya Mittal",
  "email": "bhavya@gmail.com",
  "eventId": "69c915c9e2dfc767e86d5ed9",
  "status": "registered",
  "registeredAt": "2026-03-29T12:16:29.9578531Z" }
```

7. MODULE 4 – BUDGET TRACKING

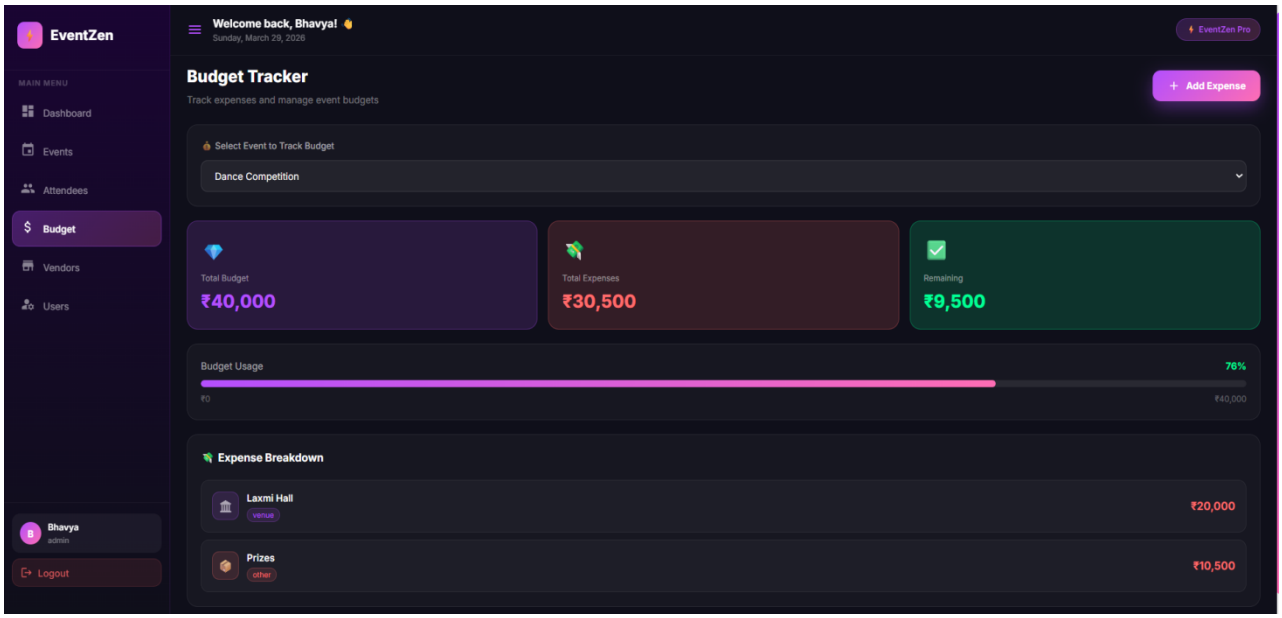
7.1 Module Overview

The Budget Tracking Module is implemented as a Spring Boot 3.x / Java 21 microservice – the only Java-based service in the stack, demonstrating technology diversity within the microservices architecture. The module provides comprehensive financial management for events. Each event can have exactly one associated budget document, and administrators can add multiple categorized expense entries. The module provides real-time remaining budget calculation.

7.2 Key Features

- Create a total budget document for any event with an initial budget amount
- Add expense entries to any budget with category, amount, and description
- Support eight predefined expense categories: Venue, Catering, Decoration, Entertainment, Marketing, Logistics, Staff, Other
- Real-time remaining budget calculation: totalBudget minus sum of all expense amounts
- Visual budget utilization progress bars in the React frontend (via Recharts)
- One-to-one relationship: each event has exactly one budget document
- One-to-many relationship: each budget document contains multiple expense entries

Budget Tracking Page



7.3 MongoDB Schema – Budget Collection (eventzen_budget database)

Field	Type	Required	Description
_id	ObjectId	Auto	MongoDB primary key
eventId	String	Yes	References Events._id
totalBudget	Number	Yes	Total allocated budget for the event
expenses	Array	No	Array of expense entry sub-documents
expenses[].category	String (Enum)	Yes	Venue / Catering / Decoration / Entertainment / Marketing / Logistics / Staff / Other
expenses[].amount	Number	Yes	Expense amount in currency units
expenses[].description	String	No	Expense description or note
expenses[].date	Date	Auto	Expense entry timestamp

7.4 API Endpoints – Budget Service (Base URL: http://localhost:5003)

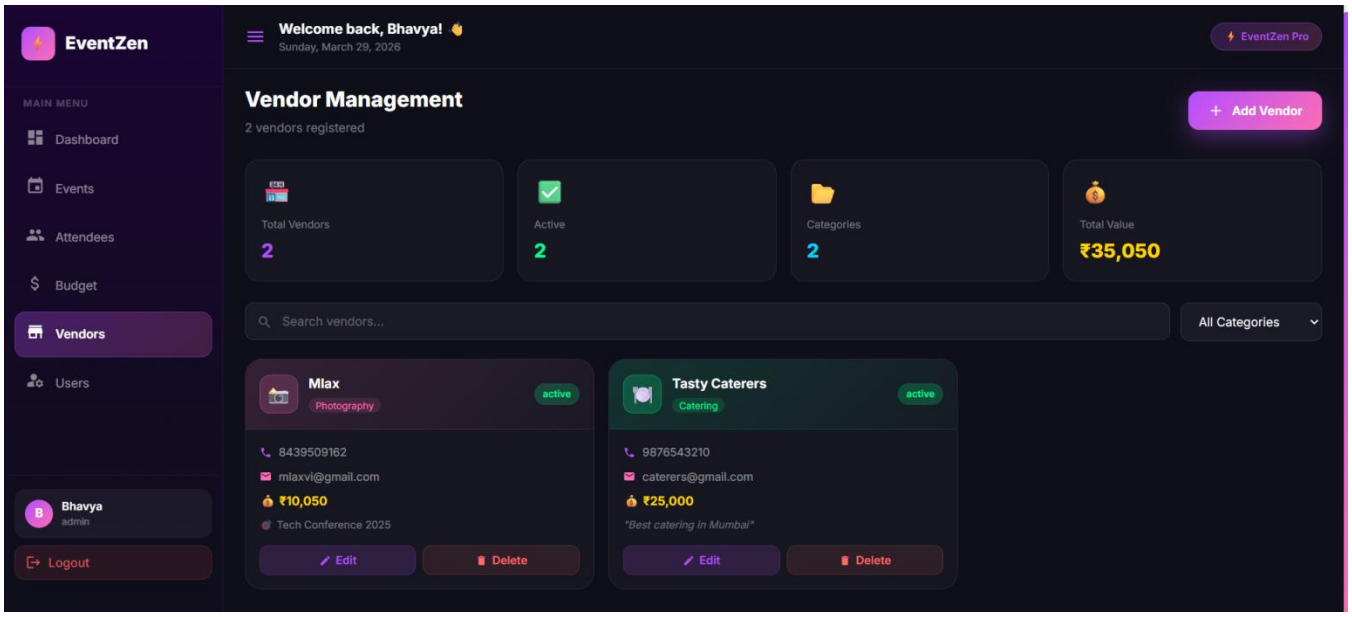
#	Method	Endpoint	Description	Auth	Code
1	POST	/api/budget	Create budget for event	JWT (Admin)	201
2	GET	/api/budget/{eventId}	Get budget & expenses by event	JWT	200
3	POST	/api/budget/{eventId}/expense	Add expense entry to budget	JWT (Admin)	200
4	PUT	/api/budget/{eventId}	Update total budget amount	JWT (Admin)	200
5	DELETE	/api/budget/{eventId}/expense/{id}	Remove specific expense entry	JWT (Admin)	200

8. MODULE 5 – VENDOR MANAGEMENT

8.1 Module Overview

The Vendor Management Module is co-located within the Event Service (Node.js + MongoDB, Port 5002). It enables administrators to build and maintain a vendor directory and associate vendors with specific events. Vendors represent external service providers hired for event execution, categorized by the type of service they offer.

Vendor Management Page



8.2 Vendor Categories

Category	Description	Example Vendors
Catering	Food and beverage services	Catering companies, restaurants, food trucks
Decoration	Event decor and theming	Florists, lighting companies, stage designers
Entertainment	Performances and entertainment	Bands, DJs, comedians, performers
Photography	Visual documentation	Photographers, videographers, photo booths
Logistics	Operations and transportation	Security, parking, transportation companies
AV/Tech	Audio-visual technology	Sound systems, projectors, streaming services

Category	Description	Example Vendors
Other	Any uncategorized service	Miscellaneous vendors

8.3 API Endpoints – Vendor Management (Base URL: http://localhost:5002)

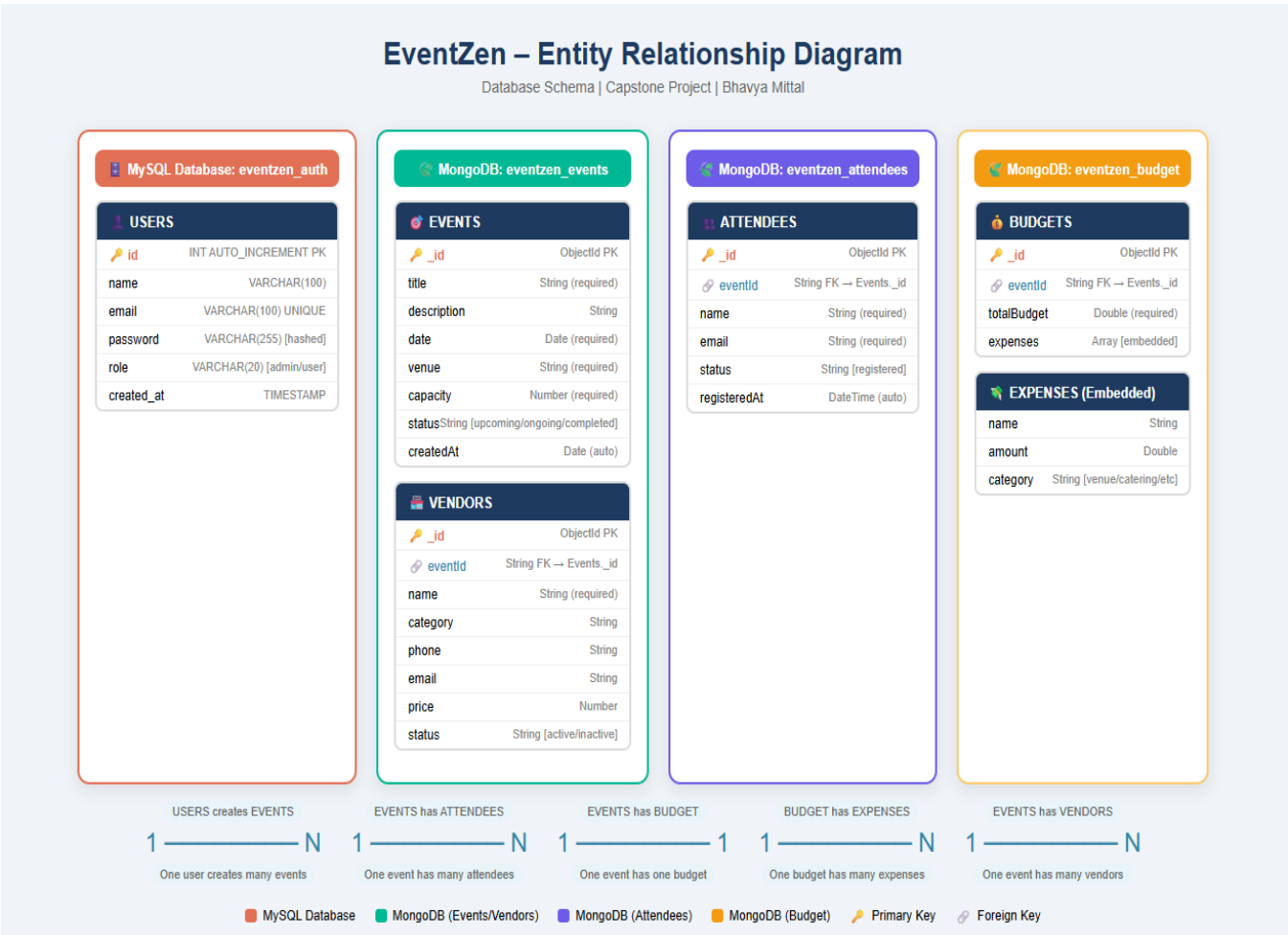
#	Method	Endpoint	Description	Auth	Code
1	GET	/api/vendors	List all vendors	No	200
2	GET	/api/vendors/:id	Get vendor by ID	No	200
3	POST	/api/vendors	Create a new vendor	JWT (Admin)	201
4	PUT	/api/vendors/:id	Update vendor details	JWT (Admin)	200
5	DELETE	/api/vendors/:id	Delete a vendor	JWT (Admin)	200

9. ER DIAGRAM & DATABASE SCHEMA

9.1 Polyglot Persistence Architecture

EventZen employs a polyglot persistence strategy using two database systems. MySQL stores authentication data requiring relational integrity, while MongoDB stores all domain data requiring schema flexibility. The ER Diagram below illustrates entity relationships across both systems.

ER Diagram



9.2 Entity Relationship Summary

Entity A	Relationship	Entity B	Cardinality	Database
User	creates	Event	1 to N	MySQL → MongoDB

Entity A	Relationship	Entity B	Cardinality	Database
Event	has	Attendees	1 to N	MongoDB → MongoDB
Event	has	Budget	1 to 1	MongoDB → MongoDB
Event	has	Vendors	1 to N	MongoDB → MongoDB
Budget	contains	Expenses	1 to N	Embedded document

9.3 MySQL Database – eventzen_auth

Column	Type	Constraint	Notes
id	INT	PK, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)	NOT NULL	User full name
email	VARCHAR(100)	UNIQUE, NOT NULL	Unique login identifier
password	VARCHAR(255)	NOT NULL	bcrypt hash (never plaintext)
role	VARCHAR(20)	DEFAULT 'user'	Values: 'admin' or 'user'
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Auto-populated on insert

9.4 MongoDB Databases & Collections

Database	Collection	Primary Key	Key Fields
eventzen_events	events	_id (ObjectId)	title, date, venue, capacity, status, createdAt
eventzen_events	vendors	_id (ObjectId)	eventId, name, category, price, phone
eventzen_attendees	attendees	_id (ObjectId)	eventId, name, email, status, registeredAt
eventzen_budget	budgets	_id (ObjectId)	eventId, totalBudget, expenses[]

10. COMPLETE API ENDPOINTS REFERENCE

10.1 Auth Service – http://localhost:5001

#	Method	Endpoint	Request Body Fields	Auth	Response
1	POST	/api/auth/register	name, email, password, role, adminCode*	No	201 – {message}
2	POST	/api/auth/login	email, password	No	200 – {token, user}
3	GET	/api/auth/users	–	JWT (Admin)	200 – users[]

** adminCode required only when role = 'admin'*

10.2 Event Service – http://localhost:5002

#	Method	Endpoint	Description	Auth	Code
1	GET	/api/events	List all events	No	200
2	GET	/api/events/:id	Get event by MongoDB ObjectId	No	200
3	POST	/api/events	Create event (title, date, venue, capacity, status)	JWT	201
4	PUT	/api/events/:id	Update event fields by ID	JWT	200
5	DELETE	/api/events/:id	Permanently delete event	JWT	200
6	GET	/api/vendors	List all vendors	No	200
7	GET	/api/vendors/:id	Get vendor by ID	No	200
8	POST	/api/vendors	Create vendor (name, category, eventId, price)	JWT	201
9	PUT	/api/vendors/:id	Update vendor details	JWT	200
10	DELETE	/api/vendors/:id	Delete vendor	JWT	200

10.3 Attendee Service – http://localhost:5172

#	Method	Endpoint	Description	Auth	Code
1	GET	/api/attendee/event/{eventId}	Get all attendees for specified event	No	200
2	POST	/api/attendee	Register attendee (name, email, eventId)	No	200
3	DELETE	/api/attendee/{id}	Remove attendee by ID	JWT	200

10.4 Budget Service – http://localhost:5003

#	Method	Endpoint	Description	Auth	Code
1	POST	/api/budget	Create budget for event (eventId, totalBudget)	JWT	201
2	GET	/api/budget/{eventId}	Get budget with all expenses for event	JWT	200
3	POST	/api/budget/{eventId}/expense	Add expense (category, amount, description)	JWT	200
4	PUT	/api/budget/{eventId}	Update total budget amount	JWT	200
5	DELETE	/api/budget/{eventId}/expense/{id}	Remove specific expense entry	JWT	200

10.5 HTTP Status Code Reference

Status Code	Description	Usage in EventZen
200 OK	Request successful	Successful GET, PUT, DELETE operations
201 Created	Resource created	Successful POST – new user, event, vendor, budget
400 Bad Request	Invalid input	Missing fields, wrong password, validation failure
401 Unauthorized	Authentication failed	Missing or invalid JWT token
403 Forbidden	Insufficient permissions	Valid token but wrong role
404 Not Found	Resource not found	Event/attendee/vendor ID does not exist
409 Conflict	Duplicate resource	Email already registered, duplicate attendee booking
500 Internal Server Error	Server-side error	Unhandled exceptions, database connection failures

11. FRONTEND PAGES & USER INTERFACE

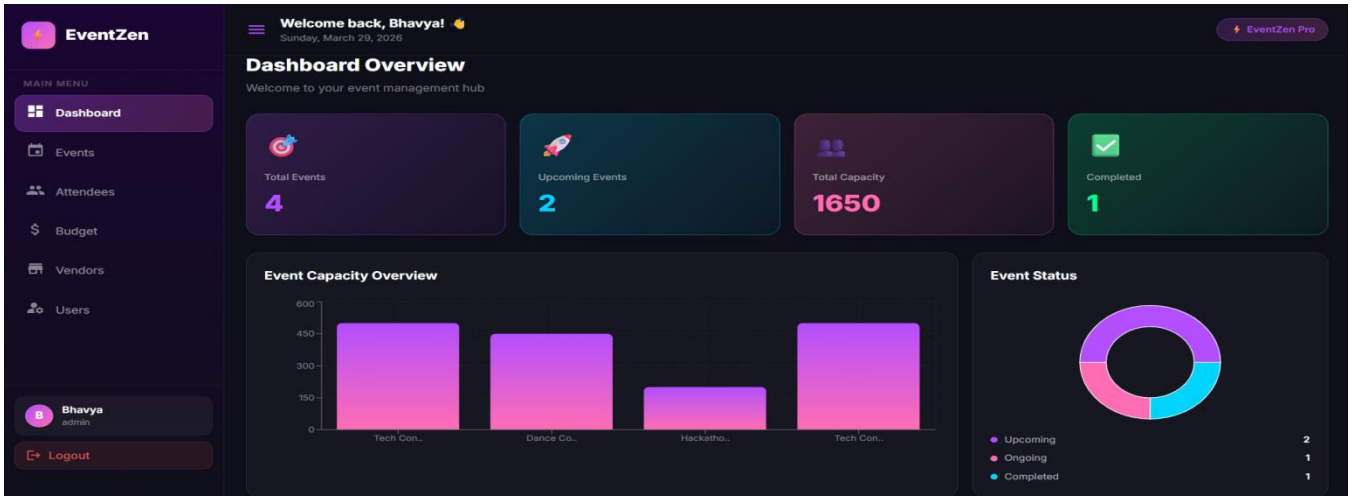
11.1 Frontend Technology

The EventZen frontend is a React.js 18 Single-Page Application (SPA) served on Port 3000. It features two completely separate portals with dedicated layouts: the Admin Portal with a sidebar navigation for management operations, and the Customer Portal with a top navigation bar for event discovery and booking. The UI uses Framer Motion for animations and Recharts for data visualization on the admin dashboard.

11.2 Admin Portal – Page Reference

Page	URL Route	Description	Key Features
Login	/login	Authentication entry point	Email/password form, role detection, portal redirect
Register	/register	New account creation	Role dropdown, admin secret code field, validation
Dashboard	/	Admin overview	KPI stats cards, Recharts bar chart, pie chart, activity log
Events	/events	Event CRUD	Create/Edit/Delete events, status toggle, search & filter
Attendees	/attendees	Attendee management	Filter by event, attendee list table, remove action
Budget	/budget	Financial tracking	Create budget, add expenses, category dropdown, progress bar
Vendors	/vendors	Vendor directory	Add/Edit/Delete vendors, category filter, event assignment
Users	/users	User management	View all registered users, roles, creation dates

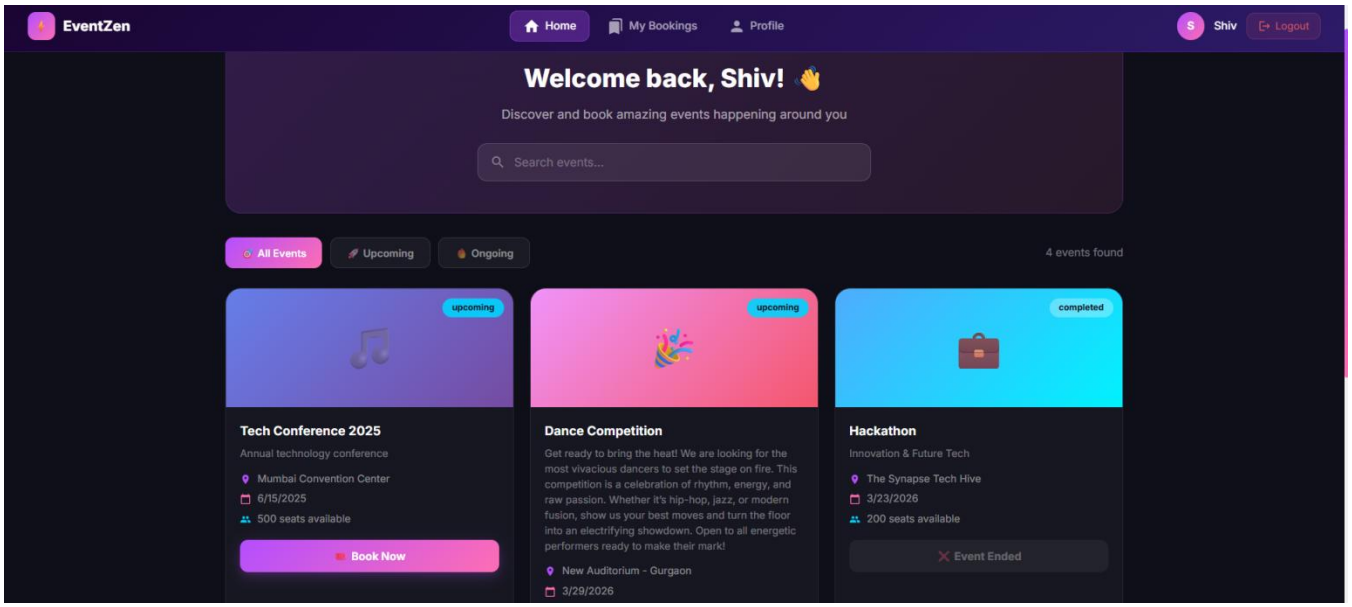
Admin Portal



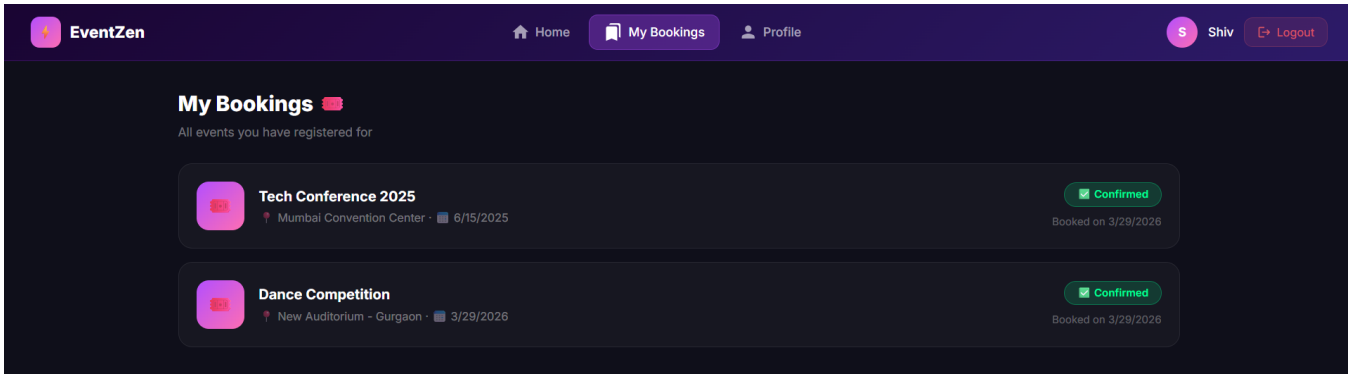
11.3 Customer Portal – Page Reference

Page	URL Route	Description	Key Features
Home	/user	Event discovery & booking	Search bar, status filter (All/Upcoming/Ongoing), event cards with Book Now
My Bookings	/user/bookings	Personal booking history	Confirmed bookings list, event details, venue, date, booking date
Profile	/user/profile	Account information	Name, email, role, account creation timestamp

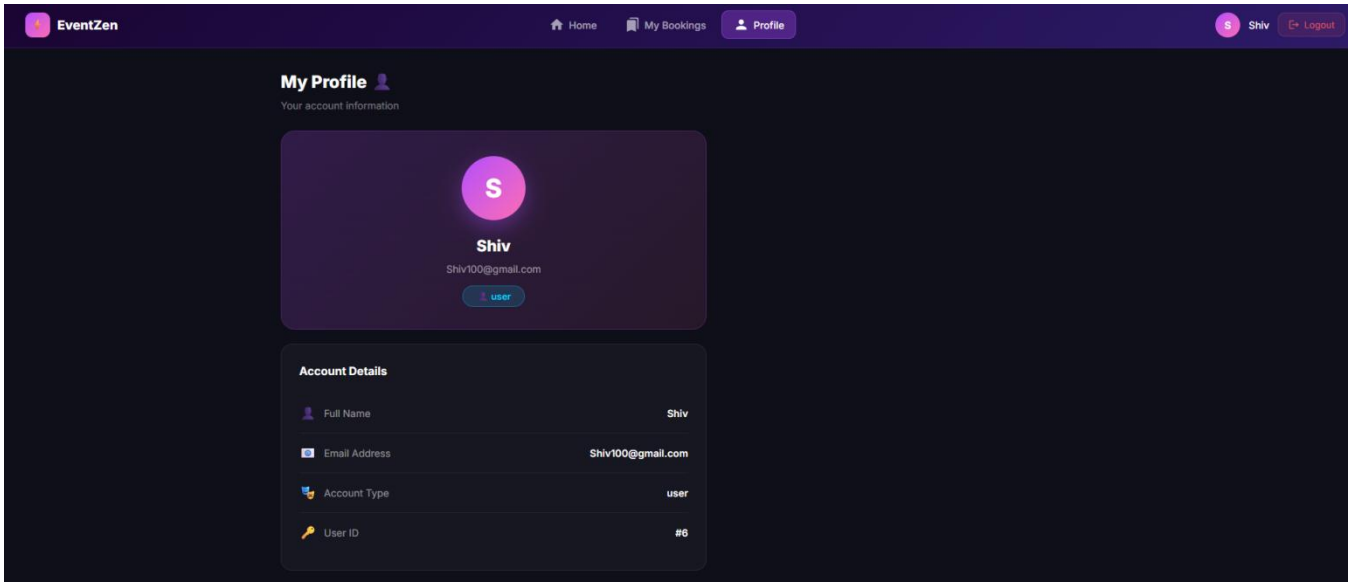
User Home Page



My Bookings Page



Profile Page



11.4 Frontend Components Structure

Component/File	Location	Purpose
Layout.js	src/components/	Admin portal wrapper – sidebar navigation, page structure
UserLayout.js	src/components/	Customer portal wrapper – top navbar, logout button
AuthContext.js	src/context/	Global authentication state management via React Context API
api.js	src/services/	Axios base URL configuration for all service endpoints
Dashboard.js	src/pages/	Admin dashboard with KPI cards and Recharts charts
Events.js	src/pages/	Event management CRUD interface
Attendees.js	src/pages/	Attendee management with event filter
Budget.js	src/pages/	Budget creation and expense tracking
Vendors.js	src/pages/	Vendor management directory
Users.js	src/pages/	User listing for admin oversight
UserHome.js	src/pages/user/	Customer event browsing and booking

Component/File	Location	Purpose
UserBookings.js	src/pages/user/	Customer's registered event history
UserProfile.js	src/pages/user/	Customer account information
Login.js	src/pages/	Shared login page for both portals
Register.js	src/pages/	Account registration with role selection

12. USER FLOW DIAGRAMS

12.1 Admin User Flow

```
START
  |
  v
Navigate to /login  ──────── New user ──────> /register
  |                                              |
  | Enter email + password                       | Name, Email, Password,
  v                                              | Role = Admin, Admin Code
[JWT Token Issued]                               v
  |                                     [Admin Account Created]
  v                                              |
[Admin Dashboard /]  <──────────────────────────+
  |
  |──> /events   → Create / Edit / Delete / Change Status
  |──> /attendees → Filter by Event / View Roster / Remove
  |──> /budget   → Set Budget / Add Expenses / Track Spending
  |──> /vendors  → Add / Edit / Delete / Assign to Event
  |──> /users    → View All Registered Users
  |
  v
[Logout] → Token Cleared → Redirect to /login
  |
END
```

12.2 Customer User Flow

```
START
  |
  v
Navigate to /login  ──────── New user ──────> /register
  |                                              |
  | Enter email + password                       | Name, Email, Password,
  v                                              | Role = User (no code needed)
[JWT Token Issued]                               v
  |                                    [User Account Created]
  v                                              |
[Customer Home /user]  <──────────────────────── +
  |
  |──> Browse Events (cards grid with status badges)
  |──> Search Events by keyword
  |──> Filter: All Events / Upcoming / Ongoing
```

```
    |
    |──> Click event card  →  View event details
    |       |
    |       └──> Click 'Book Now'
    |               |
    |               v
    |         [POST /api/attendee with name + email + eventId]
    |               |
    |               v
    |         [Booking Confirmed]
    |
    |──> /user/bookings   → View all registered events (Confirmed status)
    |──> /user/profile    → View account info
    |
   v
[Logout] → Token Cleared → Redirect to /login
END
```

12.3 Authentication Flow – Step by Step

Step	Actor	Action	System Response
1	User	Navigate to /register	Render registration form with role dropdown
2	User	Fill and submit registration form	POST /api/auth/register with payload
3	Auth Service	Validate all required fields	Check email uniqueness in MySQL
4	Auth Service	Hash password with bcrypt	Store user record in MySQL with hashed password
5	System	Return HTTP 201 Created	Redirect user to /login page
6	User	Enter email and password	POST /api/auth/login with credentials
7	Auth Service	Retrieve user by email from MySQL	Compare submitted password with bcrypt hash
8	Auth Service	On match, sign JWT with userId and role	Return JWT token + user object to client
9	React Frontend	Store token in localStorage	Decode role from JWT payload
10	React Frontend	Redirect based on role	Admin → / (Dashboard) | User → /user (Home)
11	User	Access protected page	JWT sent in Authorization: Bearer header on every request
12	JWT Middleware	Verify token signature and expiry	Allow request or return 401 Unauthorized

13. DOCKER & DEPLOYMENT CONFIGURATION

13.1 Docker Architecture Overview

EventZen is fully containerized using Docker and orchestrated with Docker Compose version 3.8. All 7 services run in isolated containers within a shared Docker bridge network named eventzen-network. Data is persisted across container restarts through named Docker volumes for both MongoDB (mongodb_data) and MySQL (mysql_data).

13.2 Docker Services Configuration

Service	Build Context	Container	Host Port	Depends On
mongodb	Image: mongo:6.0	eventzen-mongodb	27017:27017	–
mysql	Image: mysql:8.0	eventzen-mysql	3307:3306	–
auth-service	./auth-service	eventzen-auth	5001:5001	mysql (healthy)
event-service	./event-service	eventzen-events	5002:5002	mongodb (healthy)
attendee-service	./attendee-service/AttendeeService	eventzen-attendees	5172:8080	mongodb (healthy)
budget-service	./budget-service	eventzen-budget	5003:5003	mongodb (healthy)
frontend	./frontend	eventzen-frontend	3000:3000	All services

13.3 Environment Variables – Complete Reference

Service	Variable	Value	Purpose
auth-service	DB_HOST	mysql	MySQL container DNS name
auth-service	DB_USER	root	MySQL username
auth-service	DB_PASSWORD	root1234	MySQL root

Service	Variable	Value	Purpose
			password
auth-service	DB_NAME	eventzen_auth	Target MySQL database
auth-service	JWT_SECRET	eventzen_secret_key_2025	JWT signing secret key
auth-service	ADMIN_CODE	EVENTZEN@ADMIN2025	Admin registration secret
auth-service	PORT	5001	Service listen port
event-service	MONGO_URI	mongodb://mongodb:27017/eventzen_events	MongoDB connection string
event-service	PORT	5002	Service listen port
attendee-service	MongoDBSettings__ConnectionString	mongodb://mongodb:27017	MongoDB host connection
attendee-service	MongoDBSettings__DatabaseName	eventzen_attendees	Target MongoDB database
attendee-service	MongoDBSettings__CollectionName	attendees	Target MongoDB collection
budget-service	SPRING_DATA_MONGODB_URI	mongodb://mongodb:27017/eventzen_budget	MongoDB connection string
mysql	MYSQL_ROOT_PASSWORD	root1234	MySQL root account password
mysql	MYSQL_DATABASE	eventzen_auth	Auto-create database on startup

13.4 Docker Commands – Quick Reference

Command	Description
git clone https://github.com/Bhavya333-web/eventzen.git && cd eventzen	Clone repository

Command	Description
docker-compose up --build	Build all images and start all 7 containers (first run)
docker-compose up	Start containers without rebuilding (subsequent runs)
docker-compose up -d	Start containers in detached (background) mode
docker-compose down	Stop and remove all containers
docker-compose down -v	Stop containers AND remove all data volumes (reset databases)
docker ps	View all running containers with status
docker logs eventzen-auth	View real-time logs for auth service
docker logs eventzen-events	View real-time logs for event service

13.5 Health Checks Configuration

Service	Health Check Command	Interval	Timeout	Retries
MongoDB	mongosh --eval "db.adminCommand('ping')"	10s	5s	5
MySQL	mysqladmin ping -h localhost -u root -proot1234	10s	5s	10

Health checks ensure that auth-service, event-service, attendee-service, and budget-service only start after their respective databases are fully initialized and accepting connections. Without this, services would crash on startup if databases were not yet ready.

14. SECURITY IMPLEMENTATION

14.1 Security Architecture Overview

EventZen implements multiple layers of security across authentication, authorization, data protection, and network isolation. The security architecture follows industry best practices for web application security.

Security Mechanism	Implementation	Where Applied
JWT Authentication	jsonwebtoken library, signed tokens with secret key	All protected API endpoints
Password Hashing	bcrypt with salt rounds	User registration and login in MySQL
Role-Based Access Control	JWT payload contains role, middleware validates	Admin vs User portal routes
Admin Secret Code	Special code EVENTZEN@ADMIN2025 required for admin registration	Registration endpoint
Password Validation	Min 6 chars with at least 1 numeric digit	Both client-side and server-side
Email Uniqueness	MySQL UNIQUE constraint on email column	Registration prevents duplicates
Input Validation	Required field checks, type validation before processing	All POST/PUT endpoints
Network Isolation	Docker bridge network, only necessary ports exposed to host	All containers
Duplicate Booking Prevention	Email + eventId uniqueness check in attendee service	POST /api/attendee endpoint

14.2 JWT Token Structure

Each JWT token issued by the Auth Service contains the following claims:

```
Header:  { "alg": "HS256", "typ": "JWT" }
Payload: { "id": <userId>, "role": "admin"|"user", "iat": <issued_at>, "exp": <expiry> }
Signature: HMACSHA256(base64(header) + "." + base64(payload), JWT_SECRET)
```

14.3 Protected vs Public Endpoints

Access Level	Endpoints	Requirement
Public (No Auth)	GET /api/events, GET /api/vendors, POST /api/auth/register, POST /api/auth/login, GET /api/attendee/event/:id, POST /api/attendee	No token required
Authenticated (Any Role)	GET /api/budget/:eventId, GET /api/auth/users	Valid JWT required in Authorization header
Admin Only	POST/PUT/DELETE /api/events, POST/PUT/DELETE /api/vendors, POST/PUT/DELETE /api/budget/*, DELETE /api/attendee/:id	Valid JWT with role='admin'

15. TESTING & QUALITY ASSURANCE

15.1 API Testing – Postman

All API endpoints were systematically tested using Postman with both positive (expected success) and negative (expected failure/error) test cases. The following test scenarios were executed and verified:

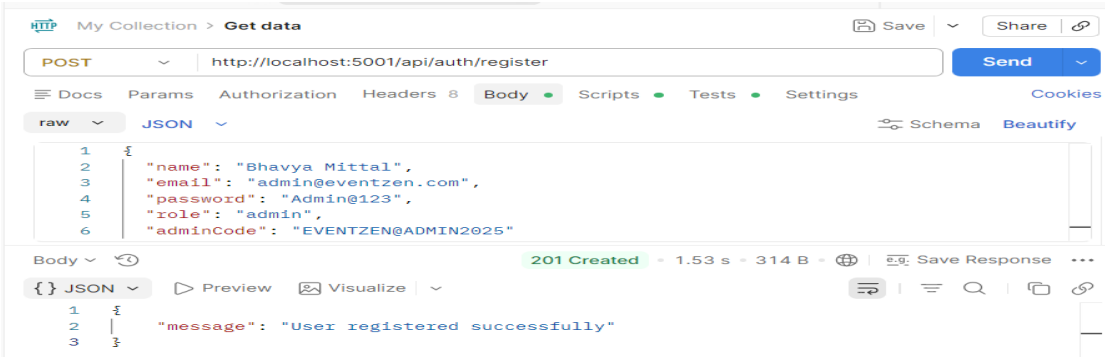
Auth Service Tests

Test Case	Method + Endpoint	Input	Expected	Result
Admin Registration	POST /api/auth/register	Valid admin data + correct code	201 Created	PASS
User Registration	POST /api/auth/register	Valid user data, no code	201 Created	PASS
Wrong Admin Code	POST /api/auth/register	Admin role + wrong code	400 Bad Request	PASS
Duplicate Email	POST /api/auth/register	Already registered email	409 Conflict	PASS
Valid Login	POST /api/auth/login	Correct credentials	200 OK + JWT token	PASS
Wrong Password	POST /api/auth/login	Correct email, wrong password	400 Bad Request	PASS
Non-existent User	POST /api/auth/login	Unregistered email	404 Not Found	PASS
Get All Users	GET /api/auth/users	Valid admin JWT	200 OK + users[]	PASS

TEST 1 — Register Admin

- Method: **POST**
- URL: `http://localhost:5001/api/auth/register`
- Body → raw → JSON:

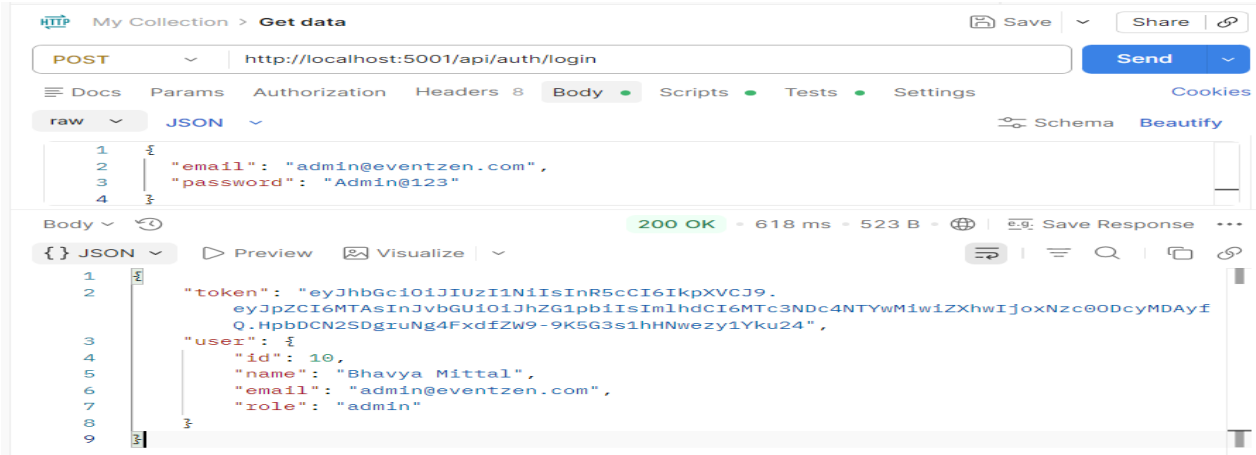
json
```
{
  "name": "Bhavya Mittal",
  "email": "admin@eventzen.com",
  "password": "Admin@123",
  "role": "admin",
  "adminCode": "EVENTZEN@ADMIN2025"
}
```



TEST 2 — Login & Get Token

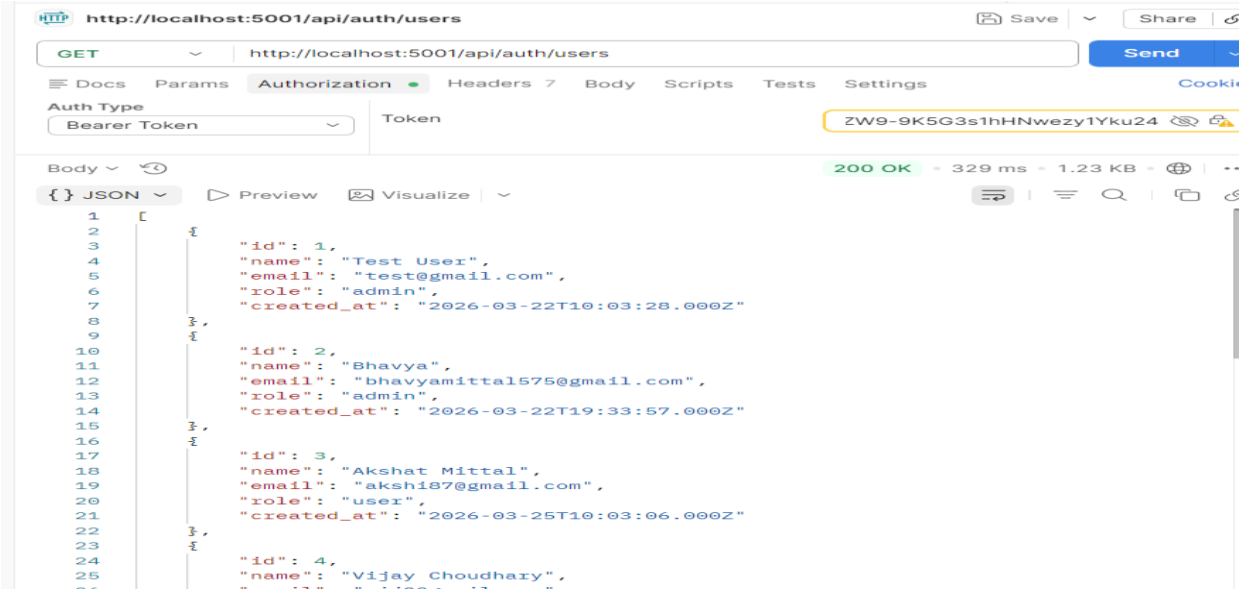
- Method: **POST**
- URL: `http://localhost:5001/api/auth/login`
- Body → raw → JSON:

```
json
{
  "email": "admin@eventzen.com",
  "password": "Admin@123"
}
```



TEST 3 — Get All Users (Protected)

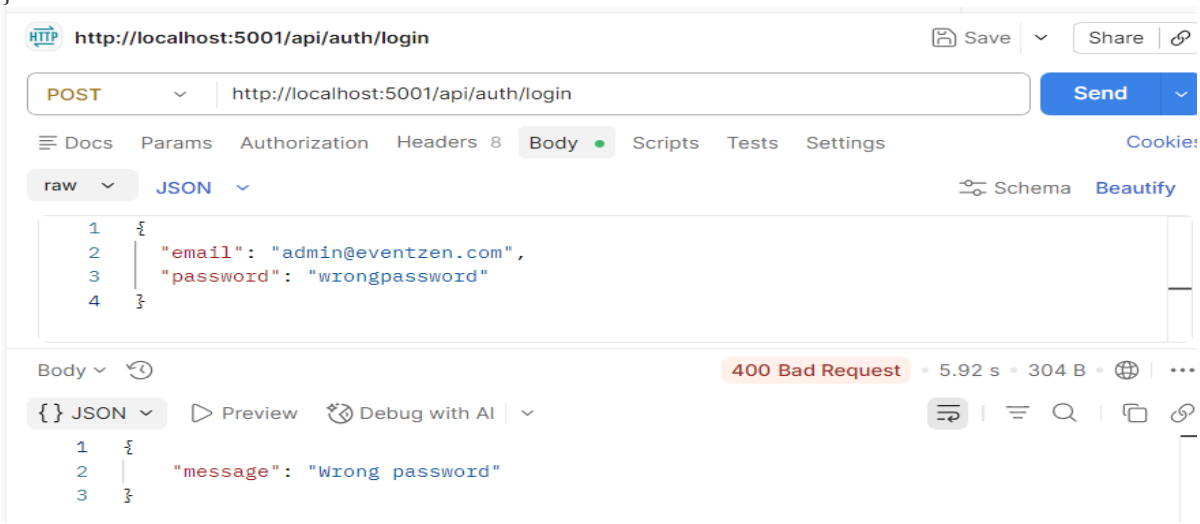
- Method: **GET**
- URL: `http://localhost:5001/api/auth/users`
- Headers tab → Add:
  - Key: `Authorization`
  - Value: `Bearer paste_your_token_here`



TEST 4 — Wrong Password (Negative Test)

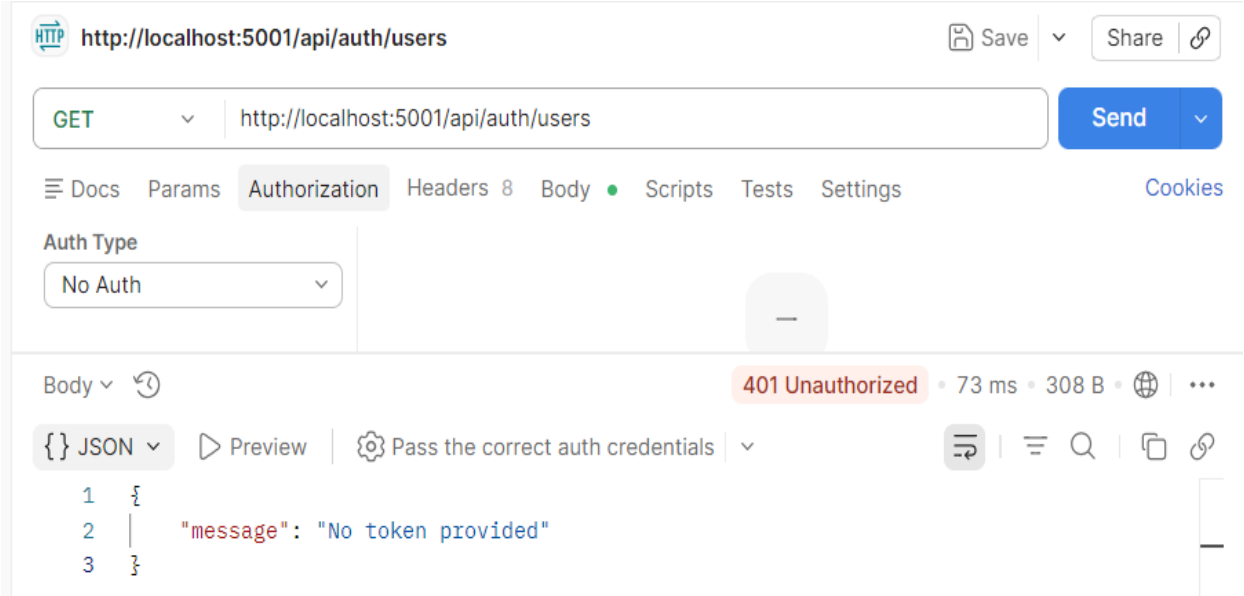
- Method: **POST**
- URL: `http://localhost:5001/api/auth/login`
- Body:

```
json
{
  "email": "admin@eventzen.com",
  "password": "wrongpassword"
}
```



TEST 5 — No Token (Negative Test)

- Method: **GET**
- URL: `http://localhost:5001/api/auth/users`
- No Authorization header



Event Service Tests

Test Case	Method + Endpoint	Expected Result
Get All Events	GET /api/events	200 OK – Returns array of event objects
Get Event by ID	GET /api/events/:id	200 OK – Returns single event document
Create Event (Auth)	POST /api/events + JWT	201 Created – New event with _id
Create Event (No Auth)	POST /api/events (no JWT)	401 Unauthorized
Update Event	PUT /api/events/:id + JWT	200 OK – Updated event object
Delete Event	DELETE /api/events/:id + JWT	200 OK – Deletion confirmed
Invalid Event ID	GET /api/events/invalid_id	400/404 – Error response

TEST 1 — Get All Events

- Method: **GET**
- URL: `http://localhost:5002/api/events`
- No auth needed

http://localhost:5002/api/events

Save | Share

GET http://localhost:5002/api/events Send

Docs | Params | Authorization | Headers 6 | Body | Scripts | Tests | Settings | Cookies

Body | 200 OK • 46 ms • 1.74 KB

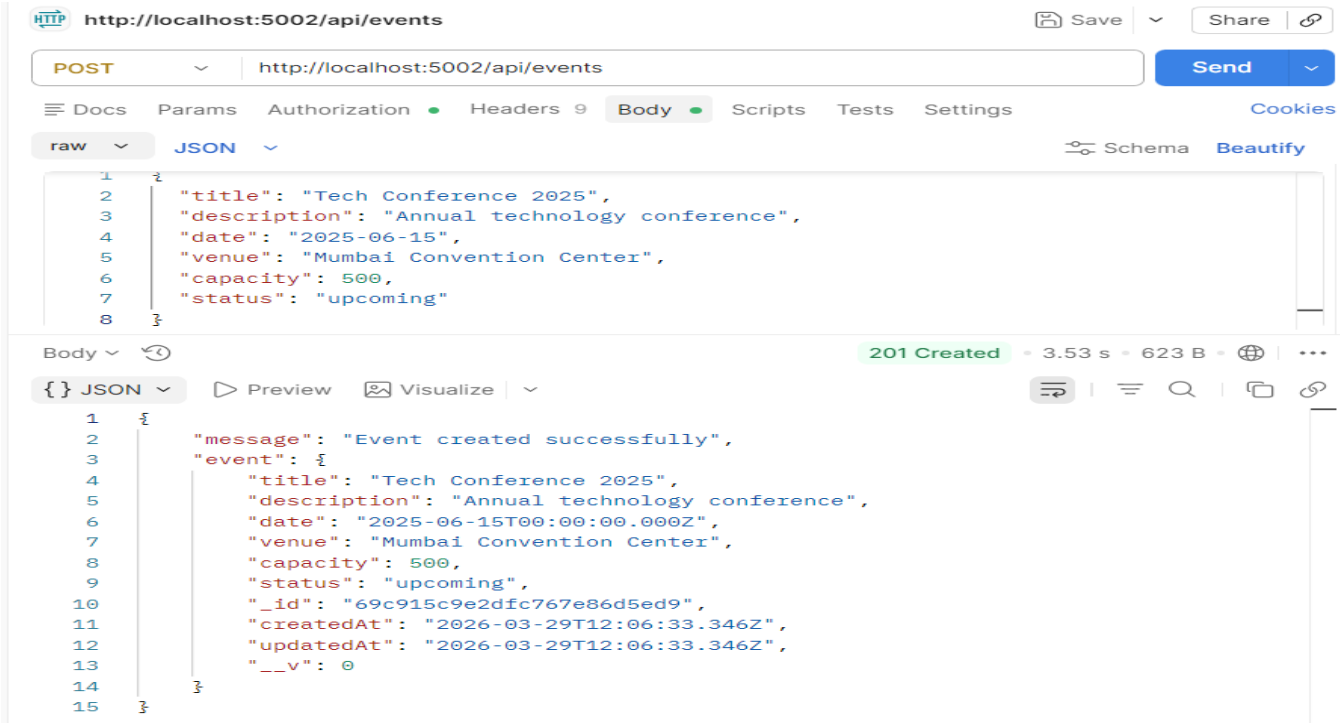
{ } JSON | Preview | Visualize


```
[
    {
        "_id": "69bfca24734ec445f33cef4e",
        "title": "Tech Conference 2025",
        "description": "Annual technology conference",
        "date": "2025-06-15T00:00:00.000Z",
        "venue": "Mumbai Convention Center",
        "capacity": 500,
        "status": "upcoming",
        "createdAt": "2026-03-22T10:53:24.660Z",
        "updatedAt": "2026-03-22T10:53:24.660Z",
        "__v": 0
    },
    {
        "_id": "69c23ea712a35a4edc6c0f44",
        "title": "Dance Competition",
        "description": "Get ready to bring the heat! We are looking for the most vivacious dancers to set the stage on fire. This competition is a celebration of rhythm, energy, and raw passion. Whether it's hip-hop, jazz, or modern fusion, show us your best moves and turn the floor into an electrifying showdown. Open to all energetic performers ready to make their mark!",
        "date": "2026-03-29T00:00:00.000Z",
        "venue": "New Auditorium - Gurgaon",
        "capacity": 450,
        "status": "upcoming",
        "createdAt": "2026-03-24T07:35:03.278Z",
        "updatedAt": "2026-03-24T07:35:20.342Z",
        "__v": 0
```


TEST 2 — Create Event

- Method: **POST**
- URL: `http://localhost:5002/api/events`
- Headers: `Authorization: Bearer your_token`
- Body → raw → JSON:

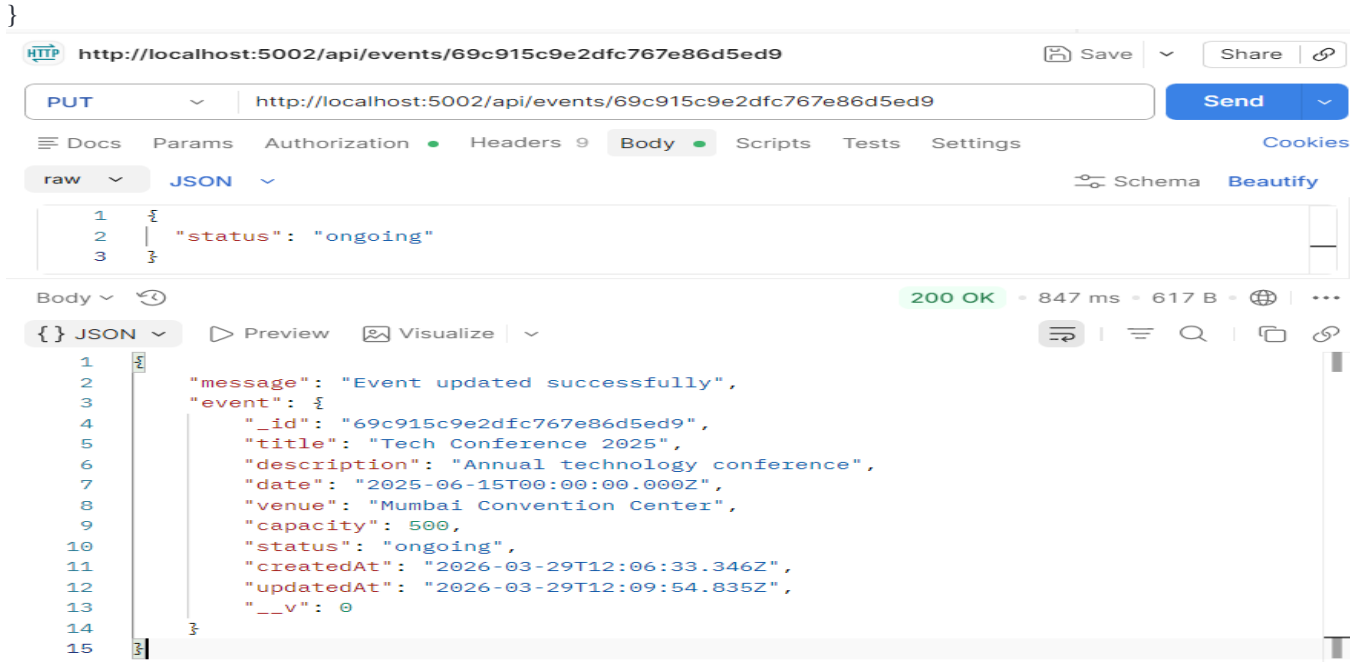
```
json
{
  "title": "Tech Conference 2025",
  "description": "Annual technology conference",
  "date": "2025-06-15",
  "venue": "Mumbai Convention Center",
  "capacity": 500,
  "status": "upcoming"
}
```



TEST 3 — Update Event

- Method: **PUT**
- URL: `http://localhost:5002/api/events/paste_event_id_here`
- Headers: `Authorization: Bearer your_token`
- Body → raw → JSON:

```
json
{
 "status": "ongoing"
}
```



Attendee Service Tests

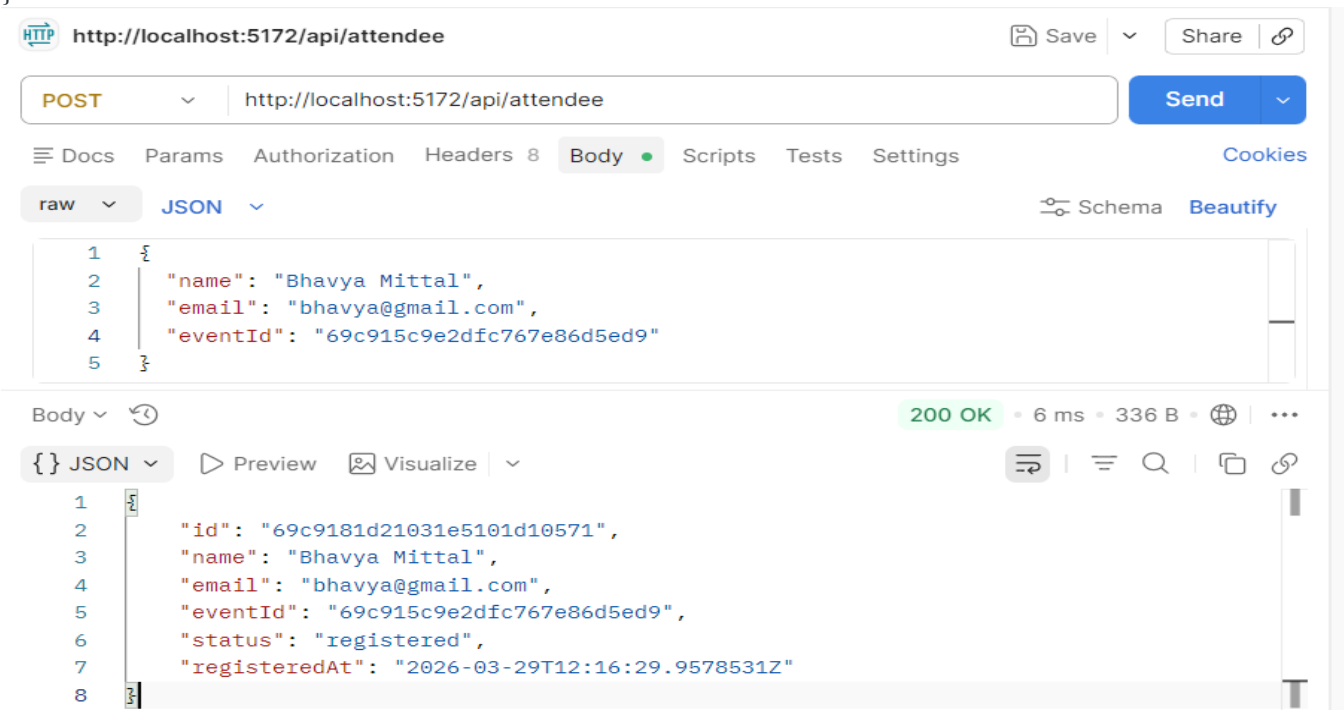
Test Case	Method + Endpoint	Expected Result
Register Attendee	POST /api/attendee	200 OK – Returns attendee object with registeredAt
Get Attendees for Event	GET /api/attendee/event/{id}	200 OK – Returns attendees array
Duplicate Registration	POST /api/attendee (same email)	409 Conflict – Duplicate prevention
Delete Attendee	DELETE /api/attendee/{id} + JWT	200 OK – Removed

TEST 1 — Register Attendee

Make sure .NET service is running! Then:

- Method: **POST**
- URL: `http://localhost:5172/api/attendee`
- Body → raw → JSON:

```
json
{
  "name": "Bhavya Mittal",
  "email": "bhavya@gmail.com",
  "eventId": "paste_event_id_here"
}
```



15.2 Frontend Testing

Test Area	Test Cases	Verification Method
Registration Form	Empty fields, weak password, wrong admin code, duplicate email	Browser validation + API error messages
Login Form	Wrong credentials, empty fields, non-existent account	Error message display
Event Booking	Book event, view in My Bookings, duplicate booking attempt	UI state verification
Navigation Flow	Role-based redirect after login, protected route access	Browser URL inspection
Event Filtering	Status filter (All/Upcoming/Ongoing), search by keyword	UI results verification
Responsive UI	Admin portal sidebar, customer portal navbar, mobile view	Browser responsive mode

16. PROJECT STRUCTURE & REPOSITORY

16.1 GitHub Repository

Field	Value
Repository URL	https://github.com/Bhavya333-web/eventzen
Repository Type	Public
Primary Language	JavaScript (90.1%)
Other Languages	C# (5.1%), Java (2.0%), HTML (1.1%), Other (1.7%)
Total Commits	63 commits
Branches	main (primary production branch)
Docker Compose	docker-compose.yml at root level
Solution File	eventzen.sln (.NET solution file at root)

16.2 Root Directory Structure

```
eventzen/
├── frontend/                    # React.js SPA – Admin + Customer portals
│   ├── src/
│   │   ├── components/          # Reusable layout components
│   │   │   ├── Layout.js        # Admin portal sidebar wrapper
│   │   │   └── UserLayout.js    # Customer portal navbar wrapper
│   │   ├── pages/               # Page-level route components
│   │   │   ├── user/            # Customer portal pages
│   │   │   │   ├── UserHome.js
│   │   │   │   ├── UserBookings.js
│   │   │   │   └── UserProfile.js
│   │   │   ├── Dashboard.js     # Admin analytics dashboard
│   │   │   ├── Events.js        # Event management CRUD
│   │   │   ├── Attendees.js     # Attendee management
│   │   │   ├── Budget.js        # Budget tracking
│   │   │   ├── Vendors.js       # Vendor management
│   │   │   ├── Users.js         # User management
│   │   │   ├── Login.js         # Shared login page
│   │   │   └── Register.js      # Account registration
│   │   ├── services/
│   │   │   └── api.js           # Axios base URL configuration
│   │   └── context/
│   │       └── AuthContext.js   # JWT auth state (React Context API)
```

```
│   └── Dockerfile
│
├── auth-service/                # Node.js + Express + MySQL (Port 5001)
│   ├── config/db.js             # MySQL connection pool
│   ├── controllers/authController.js
│   ├── routes/authRoutes.js
│   ├── middleware/authMiddleware.js  # JWT verification
│   ├── index.js
│   └── Dockerfile
│
├── event-service/               # Node.js + Express + MongoDB (Port 5002)
│   ├── models/                  # Mongoose schemas
│   │   ├── Event.js
│   │   └── Vendor.js
│   ├── controllers/
│   │   ├── eventController.js
│   │   └── vendorController.js
│   ├── routes/
│   │   ├── eventRoutes.js
│   │   └── vendorRoutes.js
│   ├── index.js
│   └── Dockerfile
│
├── attendee-service/            # .NET 8 ASP.NET Core (Port 5172)
│   └── AttendeeService/
│       ├── Controllers/AttendeeController.cs
│       ├── Attendee.cs
│       ├── AttendeeService.cs
│       ├── Program.cs
│       ├── appsettings.json
│       └── Dockerfile
│
├── budget-service/              # Spring Boot / Java 21 (Port 5003)
│   └── src/main/java/com/eventzen/budgetservice/
│       ├── Budget.java
│       ├── BudgetController.java
│       ├── BudgetRepository.java
│       ├── BudgetService.java
│       └── BudgetServiceApplication.java
│
├── □□Documentation/           # Project documentation folder
├── docker-compose.yml           # Multi-service Docker orchestration
├── eventzen.sln                 # .NET solution file
├── package.json                 # Root npm scripts (concurrently)
└── README.md                    # Project documentation
```

16.3 NPM Root Scripts

Script	Command	Description
npm run auth	cd auth-service && node index.js	Start auth service locally (without Docker)
npm run events	cd event-service && node index.js	Start event service locally (without Docker)
npm run frontend	cd frontend && npm start	Start React dev server locally
npm run dev	concurrently auth + events + frontend	Start all JS services simultaneously

PROJECT SUMMARY & ACKNOWLEDGEMENTS

Project Achievements

- 5 independent microservices built in 4 different technology stacks (Node.js, .NET 8, Spring Boot, React.js)
- Polyglot persistence: MySQL for relational data, MongoDB for document data
- Complete Docker containerization with docker-compose.yml orchestration of 7 services
- Full Role-Based Access Control with JWT authentication and bcrypt security
- Dual-portal React SPA: Admin Portal (8 pages) + Customer Portal (3 pages)
- 11 distinct module-level screens with full CRUD, search, filter, and form validation
- Comprehensive RESTful API: 20+ endpoints across 4 microservices
- Complete project documentation: ER diagram, user flows, architecture, API reference

Final Project Details

Field	Details
Project Name	EventZen – Event Management System
Submitted By	**Bhavya Mittal**
Training Program	Deloitte Full Stack Development Training Program
Trainer	**Vishwas** K Singh, SME-FSD
Submission Date	March 30, 2026
GitHub Repository	https://github.com/Bhavya333-web/eventzen
Application URL	http://localhost:3000 (after docker-compose up --build)
Admin Secret Code	EVENTZEN@ADMIN2025
Total Microservices	5 application services + 2 databases = 7 Docker containers
Total Modules	5 (Authentication, Events, Attendees, Budget, Vendors)
Total API Endpoints	20+ RESTful endpoints across 4 microservices
Total Frontend Pages	11 (8 Admin + 3 Customer)
Architecture Pattern	Microservices with Docker Compose Orchestration
Database Strategy	Polyglot Persistence – MySQL + MongoDB

Acknowledgements

This project was developed as the Capstone submission for the Deloitte Full Stack Development Training Program. I express my sincere gratitude to my Trainer **Vishwas K Singh** (SME-FSD) for his invaluable guidance, technical mentorship, and structured curriculum throughout the program. The Spring Microservices Labs document provided by the trainer was instrumental in understanding microservices patterns, Eureka service discovery, OpenFeign communication, circuit breakers, and API gateway implementation – concepts that directly informed the architectural decisions made in EventZen.

Special thanks to the Deloitte Training Team and CloudThat for designing such a comprehensive, industry-aligned full-stack curriculum covering React.js, Node.js, ASP.NET Core, Spring Boot, JWT Security, Docker, and Microservices Architecture.